YARP RV32I

Design & Verification Guide

RISC-V RV32I Processor

From ISA Fundamentals to Hardware Implementation

Key Topics:

- ✓ Computer Architecture Fundamentals
- ✓ RISC-V RV32I Instruction Set Architecture
- ✓ Instruction Formats (R, I, S, B, U, J)
- ✓ Complete ISA Reference
- ✓ Programmer's Model & Register File
- ✓ YARP Processor Microarchitecture
- ✓ RTL Implementation Walkthrough

Last Updated: April 7, 2026

Contents

1 Computer Architecture **3**
- 1.1 What Is Computer Architecture? . . . 3
- 1.2 Von Neumann Architecture . . . 3
  - 1.2.1 Key Properties . . . 4
- 1.3 Harvard Architecture . . . 4
  - 1.3.1 Key Properties . . . 5
- 1.4 Inside the CPU . . . 5
- 1.5 RISC vs. CISC . . . 6

2 The RV32I Instruction Set Architecture **8**
- 2.1 Introduction to RISC-V . . . 8
- 2.2 RV32I Overview . . . 8
- 2.3 Programmer's Model . . . 8
  - 2.3.1 The Register File . . . 8
  - 2.3.2 Register Convention (ABI) . . . 10
  - 2.3.3 Program Counter . . . 10
  - 2.3.4 Memory Model . . . 10
- 2.4 Instruction Formats . . . 12
- 2.5 R-Type Instructions (Register–Register) . . . 14
  - 2.5.1 Format . . . 14
  - 2.5.2 Fields . . . 14
  - 2.5.3 R-Type Instruction Listing . . . 14
- 2.6 I-Type Instructions (Immediate) . . . 16
  - 2.6.1 Format . . . 16
  - 2.6.2 Fields . . . 16
  - 2.6.3 I-Type Arithmetic Instructions (`opcode = 0010011`) . . . 16
  - 2.6.4 I-Type Load Instructions (`opcode = 0000011`) . . . 17
- 2.7 S-Type Instructions (Store) . . . 18
  - 2.7.1 Format . . . 18
  - 2.7.2 Why Is the Immediate Split? . . . 18
  - 2.7.3 S-Type Instruction Listing . . . 18
- 2.8 B-Type Instructions (Branch) . . . 19
  - 2.8.1 Format . . . 19
  - 2.8.2 Immediate Reconstruction . . . 19
  - 2.8.3 B-Type Instruction Listing . . . 19
- 2.9 U-Type Instructions (Upper Immediate) . . . 21
  - 2.9.1 Format . . . 21
  - 2.9.2 U-Type Instruction Listing . . . 21
- 2.10 J-Type Instructions (Jump) . . . 22
  - 2.10.1 Format . . . 22
  - 2.10.2 Immediate Reconstruction . . . 22
  - 2.10.3 Jump Instructions . . . 22

2.11 System Instructions . . . 24

3 Complete RV32I Instruction Reference **25**
3.1 Master Encoding Table . . . 25
3.2 Instruction Count Summary . . . 27

4 Instruction Details & Examples **28**
4.1 R-Type Example: ADD x5, x6, x7 . . . 28
4.2 I-Type Example: ADDI x10, x0, 42 . . . 28
4.3 S-Type Example: SW x15, 8(x2) . . . 28
4.4 B-Type Example: BEQ x1, x2, 16 . . . 28
4.5 U-Type Example: LUI x5, 0xDEADC . . . 28
4.6 J-Type Example: JAL ra, 100 . . . 29
4.7 Common Pseudoinstructions . . . 29

5 YARP RTL Implementation **30**
5.1 Step 1: Fetch . . . 30
5.1.1 Design Requirements . . . 30
5.1.2 Interface Definition . . . 31
5.1.3 RTL Implementation . . . 31
5.1.4 Cycle-by-Cycle Behavior . . . 32

1. Computer Architecture

1.1 What Is Computer Architecture?

Computer architecture describes the structure and behaviour of a computer system at a level visible to a programmer or hardware designer. It defines *how* a processor executes instructions, *how* it stores and retrieves data, and *how* the various internal components interact.

At the highest level, every computer — regardless of its complexity — follows a simple cycle:

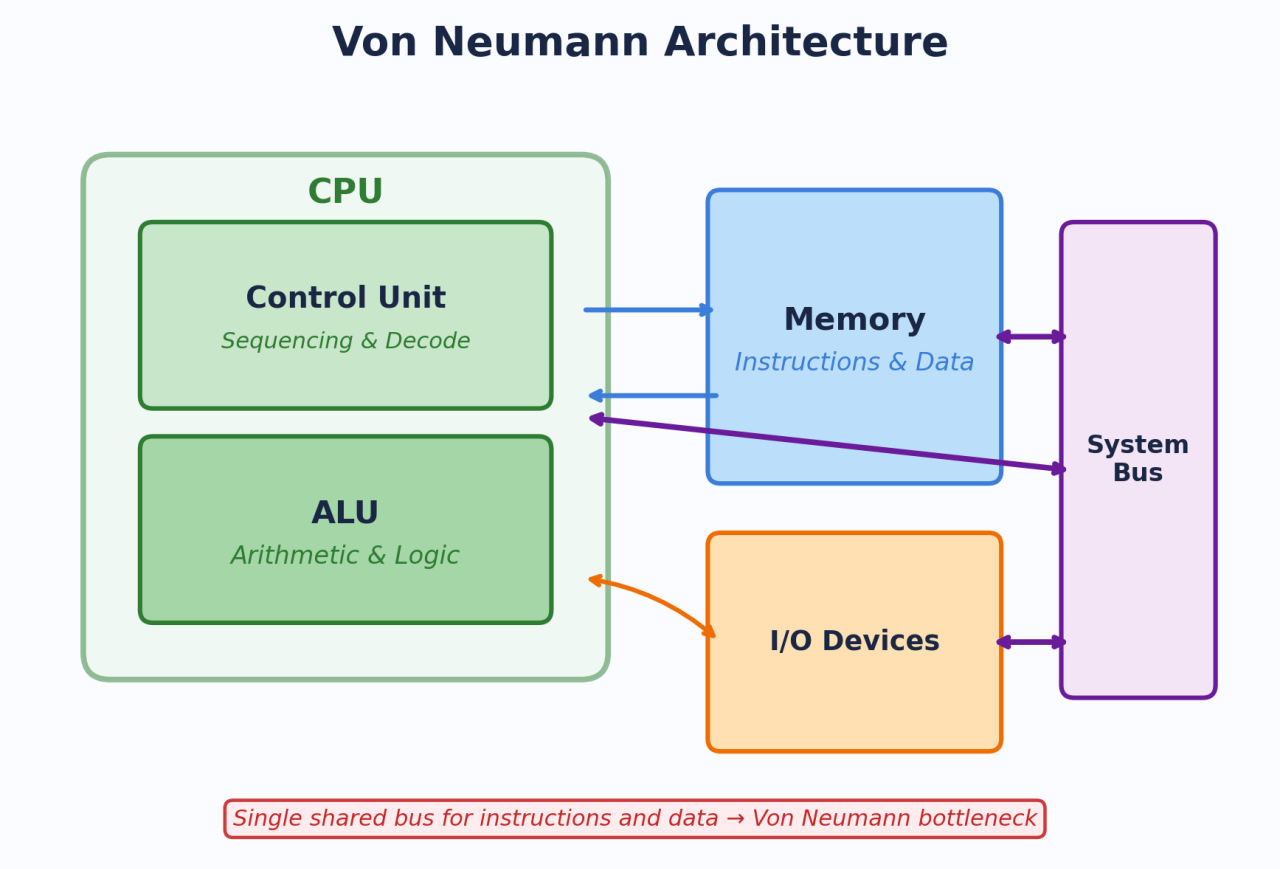
1. **Fetch** — retrieve the next instruction from memory.
2. **Decode** — interpret the instruction to determine what operation is required.
3. **Execute** — carry out the operation (arithmetic, load/store, branch, etc.).
4. **Write-back** — store the result in a register or memory.

This is commonly known as the **Fetch–Decode–Execute** cycle.

> **Key Concept:** A processor's **Instruction Set Architecture (ISA)** is the contract between software and hardware. It specifies the instructions the processor can execute, the registers available to the programmer, and the memory model. The ISA is architecture; the circuits that implement it are **microarchitecture**.

1.2 Von Neumann Architecture

The **Von Neumann architecture** (proposed by John von Neumann in 1945) is the foundational model for nearly all modern computers. Its key characteristic is a *single, shared memory* that stores both **instructions** and **data**.



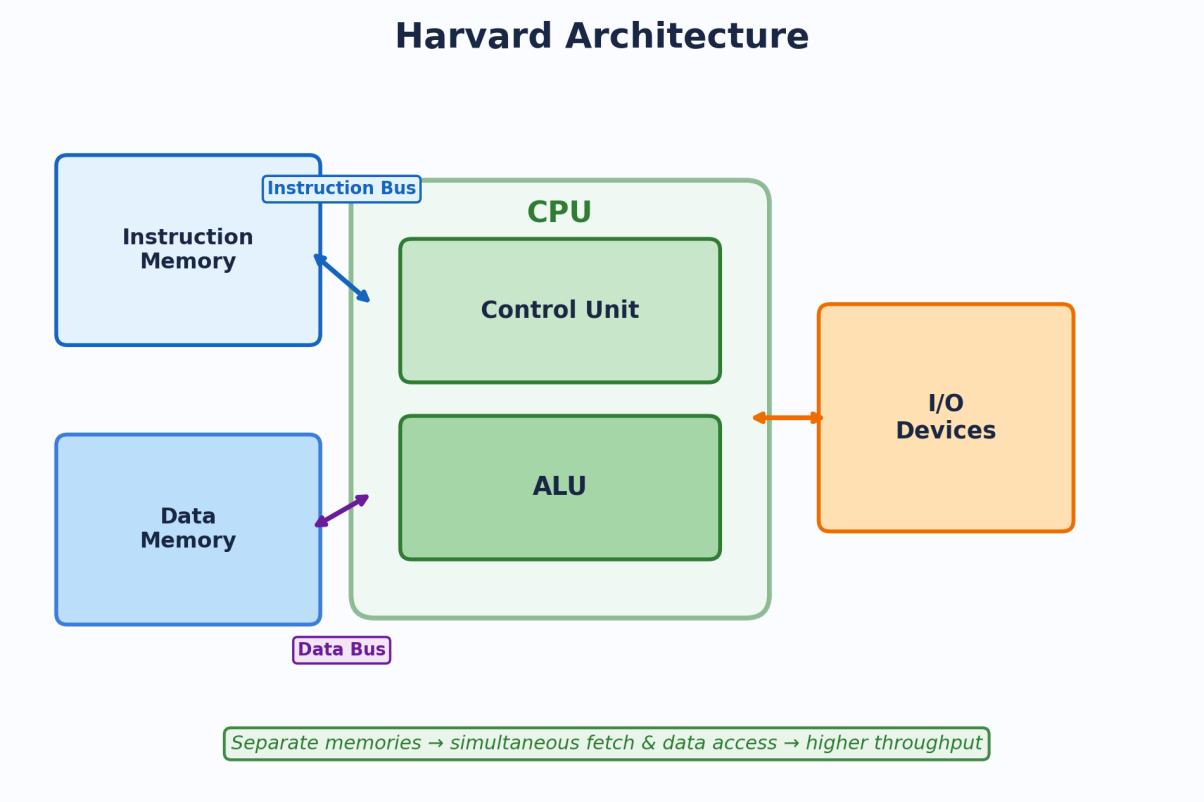
1.2.1 Key Properties

- **Single memory space:** program code and data occupy the same address range. The CPU fetches instructions and reads/writes data through the **same bus**.
- **Sequential execution:** one instruction is fetched, decoded, and executed at a time (in the simplest implementation).
- **Stored-program concept:** the program itself resides in modifiable memory, making general-purpose computing possible.

Von Neumann Bottleneck: Because instructions and data share a single bus, the CPU can either fetch an instruction *or* access data in a given cycle — never both simultaneously. This creates a bandwidth bottleneck that limits performance.

1.3 Harvard Architecture

The **Harvard architecture** solves the Von Neumann bottleneck by providing **separate memory spaces and buses** for instructions and data.



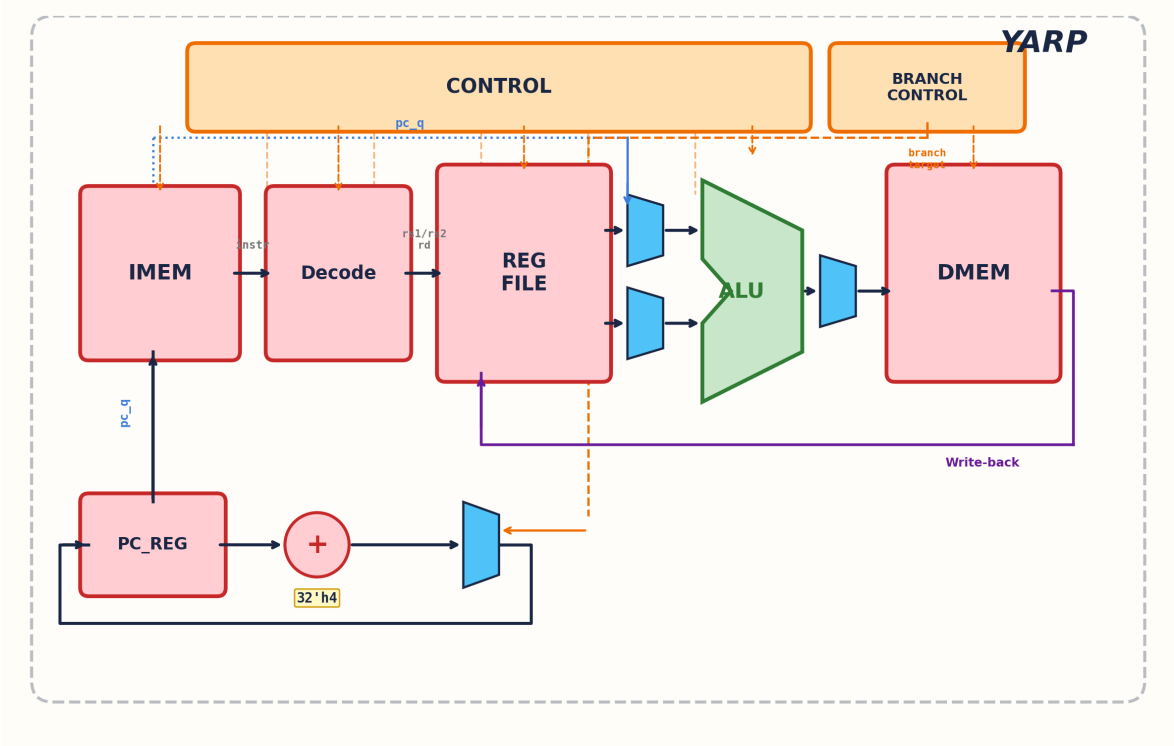
1.3.1 Key Properties

- **Separate memories:** instruction memory and data memory are physically distinct, each with its own address bus and data bus.
- **Simultaneous access:** the CPU can fetch the next instruction while reading/writing data in the same clock cycle — doubling effective bandwidth.
- **Common in embedded and RISC:** many modern processors (including RISC-V implementations) use a **Modified Harvard** architecture: separate caches for instruction and data (L1 I-cache / D-cache) backed by a unified main memory.

RISC-V and Harvard: Most practical RISC-V cores adopt a **Modified Harvard** approach — split instruction and data caches for performance, but a unified physical memory so that programs can be loaded and modified conventionally.

1.4 Inside the CPU

A CPU is built from several cooperating blocks. The diagram below shows the main components relevant to a simple RV32I implementation:



- **Program Counter (PC):** holds the address of the *current* instruction. After each fetch, the PC is incremented by 4 (the size of one RV32I instruction) or updated by a branch/jump.
- **Instruction Decoder:** breaks the 32-bit instruction word into fields (opcode, funct3, funct7, register addresses, immediate value) and determines the required operation.
- **Register File:** a bank of 32 general-purpose registers (x0–x31), each 32 bits wide. The file typically has two read ports and one write port.
- **ALU (Arithmetic Logic Unit):** performs arithmetic (add, subtract) and logic (AND, OR, XOR, shift, compare) operations.
- **Immediate Generator:** extracts and sign-extends the immediate value embedded in the instruction, rearranging the bits depending on the instruction format.
- **Control Unit:** generates control signals that steer multiplexers, the ALU operation selector, register write-enable, memory read/write, and branch decision logic.

1.5 RISC vs. CISC

Processors are broadly classified into two design philosophies:

Aspect	CISC (Complex Instruction Set)	RISC (Reduced Instruction Set)
Instruction count	Many (hundreds)	Few (typically < 50 base)
Instruction length	Variable (1–15 bytes on x86)	Fixed (4 bytes in RV32I)
Execution time	Multi-cycle per instruction	Ideally 1 cycle per instruction
Memory access	Instructions can operate directly on memory	Load/Store architecture: only `LW`/`SW` access memory
Complexity	Complex hardware decoder	Simple decoder, more compiler work
Examples	x86, x86-64	RISC-V, ARM, MIPS

Why RISC-V? RISC-V is an **open-standard** ISA released by UC Berkeley in 2010. It is modular (start with RV32I, add M, A, F, D, C extensions as needed), royalty-free, and designed for clean hardware implementation — making it an ideal educational and commercial platform.

2. The RV32I Instruction Set Architecture

2.1 Introduction to RISC-V

RISC-V (pronounced "risk-five") is a free and open Instruction Set Architecture originally developed at the University of California, Berkeley. Unlike proprietary ISAs (x86, ARM), **anyone** can implement a RISC-V processor without licensing fees.

The ISA is **modular**: a small mandatory base (`RV32I`) provides a complete integer instruction set, and optional standard extensions add functionality:

Extension	Name	Description
`I`	Base Integer	32-bit integer arithmetic, loads, stores, branches, jumps
`M`	Multiply / Divide	Hardware multiply and divide instructions
`A`	Atomic	Atomic read-modify-write for multi-core
`F`	Single-Float	IEEE 754 single-precision floating point
`D`	Double-Float	IEEE 754 double-precision floating point
`C`	Compressed	16-bit compressed instruction encoding
`Zicsr`	CSR	Control and Status Register instructions

Throughout this guide we focus exclusively on the **RV32I** base — the minimum set that every RISC-V implementation must support.

2.2 RV32I Overview

The RV32I base defines:

- **32 general-purpose registers** (`x0`–`x31`), each 32 bits wide. `x0` is hardwired to zero.
- **A 32-bit Program Counter (PC)**.
- **Fixed 32-bit instruction width** (4 bytes, naturally aligned).
- **A load/store architecture:** only load and store instructions access memory; all other operations work on registers.
- **40 base instructions** organized into 6 encoding formats.
- **Little-endian** byte ordering (with optional big-endian).
- **32-bit flat address space** ($2^{32} = 4\,\text{GiB}$).

2.3 Programmer's Model

2.3.1 The Register File

The RV32I register file contains **32 integer registers**, each 32 bits wide, designated `x0` through `x31`. In addition, each register has a *conventional ABI (Application Binary Interface) name* used by compilers and assembly programmers.

Special Register — x0 (zero): Register `x0` is **hardwired to the value 0**. Any write to `x0` is silently discarded. This simplifies many operations — for example, `addi x5, x0, 42` loads the constant 42 into `x5` without needing a separate "load immediate" instruction.

2.3.2 Register Convention (ABI)

Register	ABI Name	Role	Saver
x0	zero	Hard-wired zero	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5--x7	t0--t2	Temporaries	Caller
x8	s0/fp	Saved / frame pointer	Callee
x9	s1	Saved register	Callee
x10--x11	a0--a1	Function args / return vals	Caller
x12--x17	a2--a7	Function arguments	Caller
x18--x27	s2--s11	Saved registers	Callee
x28--x31	t3--t6	Temporaries	Caller

Caller-saved vs. Callee-saved: **Caller-saved** registers may be overwritten by any function call; the caller must save them if their values are needed later. **Callee-saved** registers must be preserved across function calls; if a function modifies them, it must save and restore them.

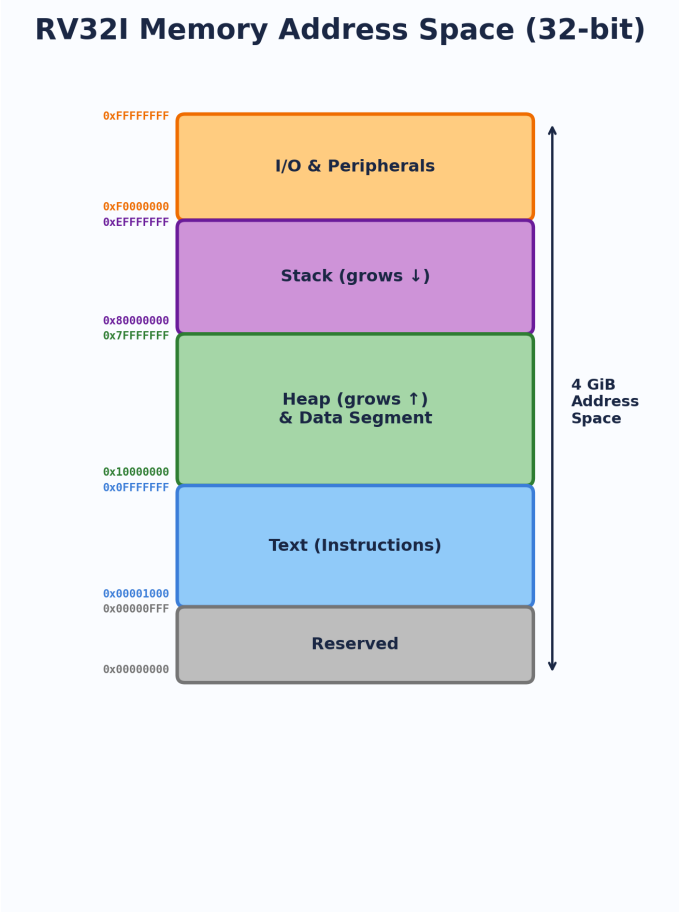
2.3.3 Program Counter

The **Program Counter (PC)** is a dedicated 32-bit register that holds the address of the *currently executing instruction*. After each instruction, the PC is updated:

- **Sequential:** $\text{PC} \leftarrow \text{PC} + 4$ (next instruction).
- **Branch taken:** $\text{PC} \leftarrow \text{PC} + \text{offset}$ (relative jump).
- **Jump:** $\text{PC} \leftarrow$ target address (absolute or register-based).

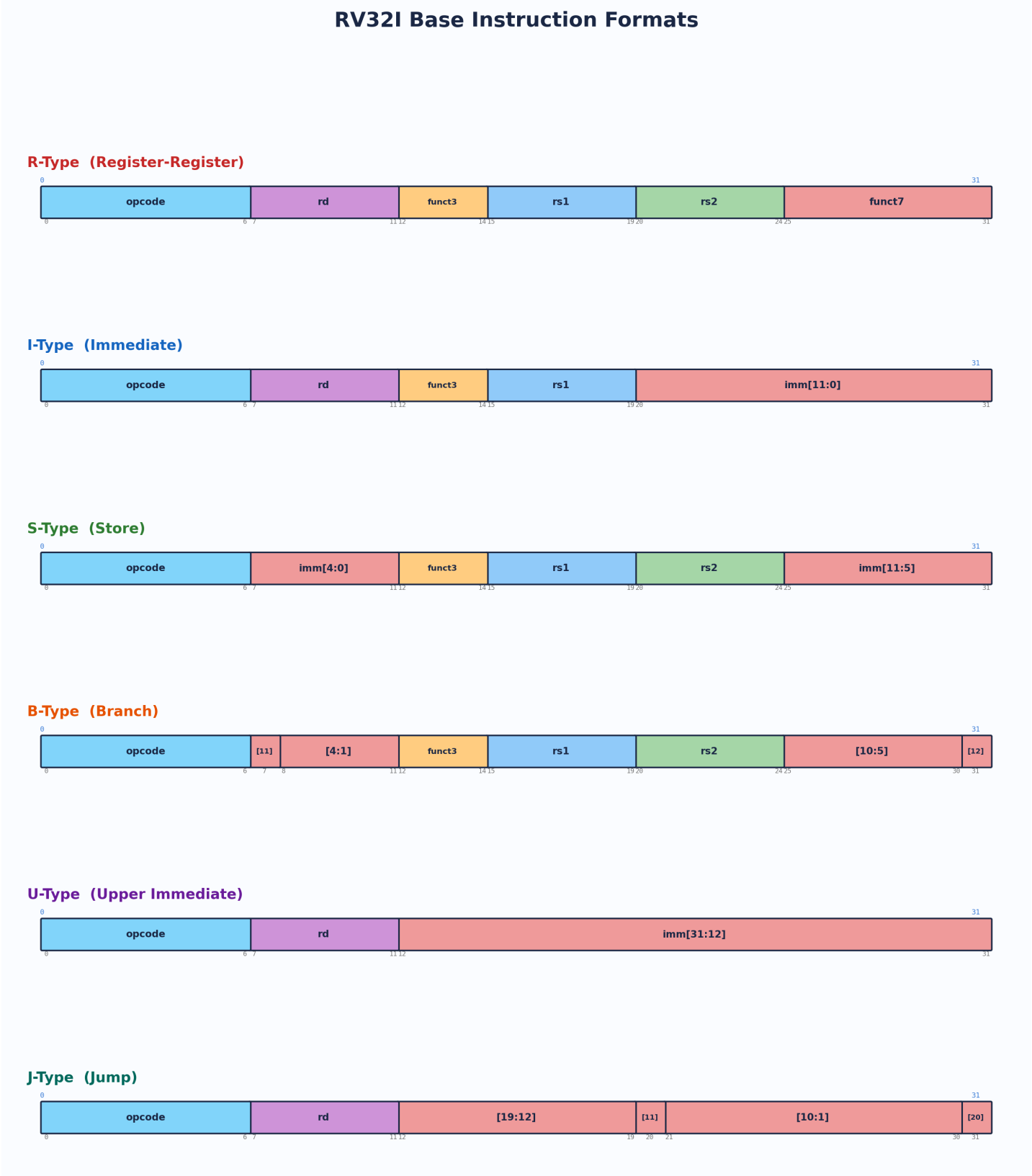
2.3.4 Memory Model

RV32I uses a **flat, byte-addressable** 32-bit address space. Memory is accessed only via load and store instructions.

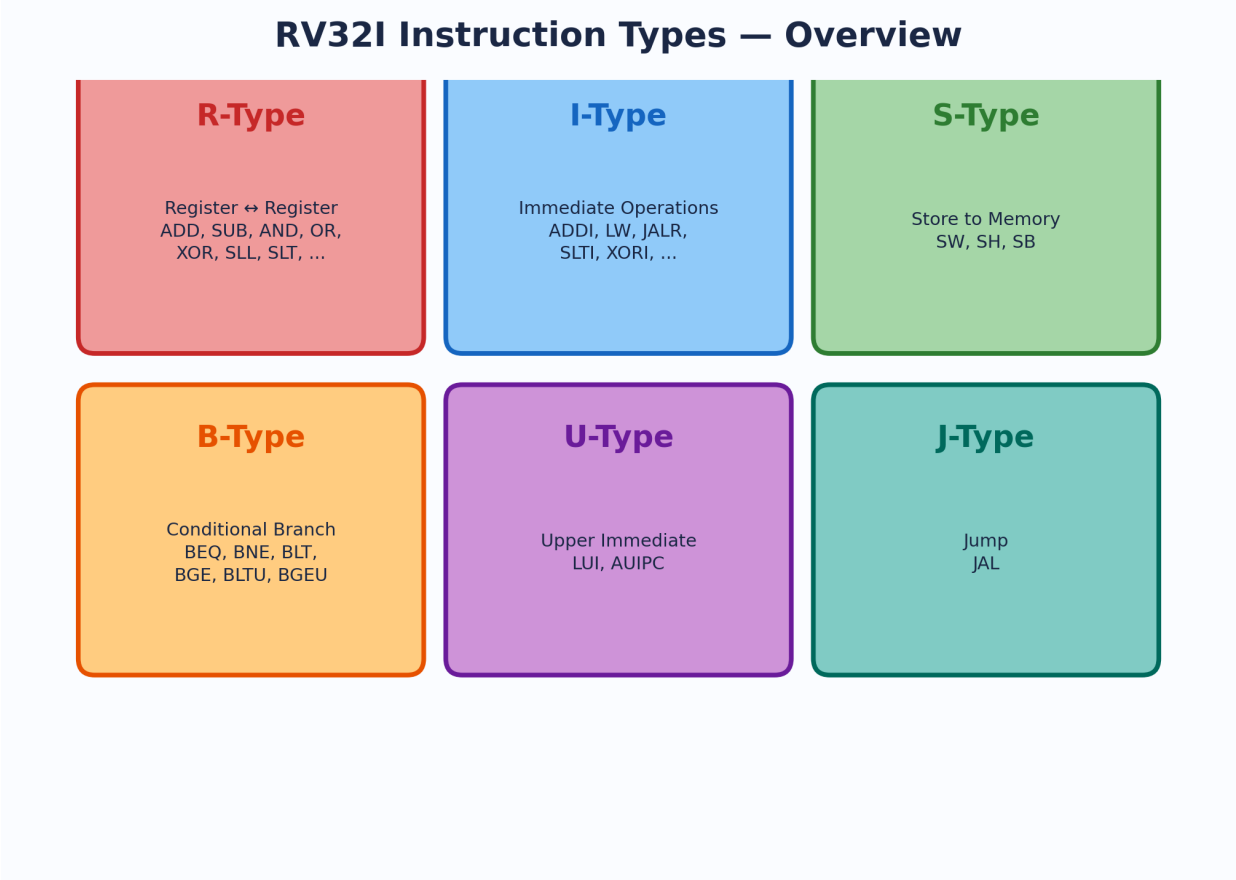

RV32I Memory Address Space (32-bit)
0xFFFFFFFF
I/O & Peripherals
0xF0000000
0xEFFFFFFF
Stack (grows ↓)
0x80000000
0x7FFFFFFF
Heap (grows ↑)
& Data Segment
0x10000000
0x0FFFFFFF
Text (Instructions)
0x00001000
0x00000FFF
Reserved
0x00000000
4 GiB
Address
Space

2.4 Instruction Formats

Every RV32I instruction is exactly **32 bits** wide. The ISA defines **six encoding formats** that specify how the 32 bits are partitioned into fields:



All formats share a common 7-bit `opcode` field in bits [6:0], which determines the general category of the instruction. The remaining 25 bits carry register addresses, function selectors, and/or immediate values.


RV32I Instruction Types — Overview
R-Type
Register ↔ Register
ADD, SUB, AND, OR,
XOR, SLL, SLT, ...
I-Type
Immediate Operations
ADDI, LW, JALR,
SLTI, XORI, ...
S-Type
Store to Memory
SW, SH, SB
B-Type
Conditional Branch
BEQ, BNE, BLT,
BGE, BLTU, BGEU
U-Type
Upper Immediate
LUI, AUIPC
J-Type
Jump
JAL

2.5 R-Type Instructions (Register–Register)

R-Type

`opcode = 0110011`

R-type instructions perform operations between **two source registers** (`rs1`, `rs2`) and write the result to a **destination register** (`rd`). No immediate value is used.

2.5.1 Format

funct7	rs2	rs1	funct3	rd	opcode
bits [31:25]	bits [24:20]	bits [19:15]	bits [14:12]	bits [11:7]	bits [6:0]
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

2.5.2 Fields

- **opcode** (7 bits): always `0110011` for R-type integer operations.
- **rd** (5 bits): destination register that receives the result.
- **funct3** (3 bits): selects the specific operation (ADD/SUB, SLL, SLT, etc.).
- **rs1** (5 bits): first source register.
- **rs2** (5 bits): second source register.
- **funct7** (7 bits): further disambiguates the operation (e.g., `0000000` for ADD vs. `0100000` for SUB).

2.5.3 R-Type Instruction Listing

Instruction	funct7	funct3	Operation
`ADD rd, rs1, rs2`	0000000	000	$\texttt{rd} \leftarrow \texttt{rs1} + \texttt{rs2}$
`SUB rd, rs1, rs2`	0100000	000	$\texttt{rd} \leftarrow \texttt{rs1} - \texttt{rs2}$
`SLL rd, rs1, rs2`	0000000	001	$\texttt{rd} \leftarrow \texttt{rs1} \ll \texttt{rs2}[4:0]$ (logical left shift)
`SLT rd, rs1, rs2`	0000000	010	$\texttt{rd} \leftarrow (\texttt{rs1} <_s \texttt{rs2})\ ?\ 1 : 0$ (signed compare)
`SLTU rd, rs1, rs2`	0000000	011	$\texttt{rd} \leftarrow (\texttt{rs1} <_u \texttt{rs2})\ ?\ 1 : 0$ (unsigned compare)
`XOR rd, rs1, rs2`	0000000	100	$\texttt{rd} \leftarrow \texttt{rs1} \oplus \texttt{rs2}$ (bitwise XOR)
`SRL rd, rs1, rs2`	0000000	101	$\texttt{rd} \leftarrow \texttt{rs1} \gg_{\text{logic}} \texttt{rs2}[4:0]$ (logical right shift)
`SRA rd, rs1, rs2`	0100000	101	$\texttt{rd} \leftarrow \texttt{rs1} \gg_{\text{arith}} \texttt{rs2}[4:0]$ (arithmetic right shift)
`OR rd, rs1, rs2`	0000000	110	$\texttt{rd} \leftarrow \texttt{rs1} \mid \texttt{rs2}$ (bitwise OR)
`AND rd, rs1, rs2`	0000000	111	$\texttt{rd} \leftarrow \texttt{rs1} \,\&\, \texttt{rs2}$ (bitwise AND)

Note: ADD and SUB share the same `funct3 = 000`. They are distinguished by `funct7`: `0000000` for ADD, `0100000` for SUB. The same pattern applies to SRL/SRA (`funct3 = 101`).

Shift reminder: A bitwise operation works directly on the 0/1 pattern stored in a register. A shift operation moves that bit pattern left or right by some number of positions.

- `SLL` **(shift left logical):** moves bits to the left, drops any bits that fall off the left side, and fills the new right-side bits with 0s. It is often used to multiply by 2^n when overflow is not a concern.
- `SRL` **(shift right logical):** moves bits to the right and fills the new left-side bits with 0s. This is useful for unsigned values and for extracting fields from an instruction or address.
- `SRA` **(shift right arithmetic):** moves bits to the right but copies the old leftmost bit (the sign bit) into the new left-side positions. This preserves the sign of a signed value.

There is no separate arithmetic left shift in RV32I: left shifts already insert 0s on the right, so `SLL` is the left-shift instruction you use.

Small example (shown in 8 bits):

$$\texttt{00101101} \ll 1 = \texttt{01011010}$$

$$\texttt{11110100} \gg_{\text{logic}} 1 = \texttt{01111010}, \qquad \texttt{11110100} \gg_{\text{arith}} 1 = \texttt{11111010}$$

If `11110100` represents -12 in two's complement, arithmetic right shift keeps the value negative, while logical right shift inserts a 0 on the left and changes the meaning.

Typical use cases: use `SLL` to scale an index by 2, 4, or 8, use `SRL` to unpack bitfields, and use `SRA` when shifting signed values right without losing the sign.

2.6 I-Type Instructions (Immediate)

I-Type

`opcodes: 0010011, 0000011, 1100111, 1110011`

I-type instructions use a **12-bit signed immediate** value as one operand alongside a source register. This format is used for **arithmetic with immediates**, **loads from memory**, **JALR**, and **system calls**.

2.6.1 Format

imm[11:0]	rs1	funct3	rd	opcode
bits [31:20]	bits [19:15]	bits [14:12]	bits [11:7]	bits [6:0]
12 bits	5 bits	3 bits	5 bits	7 bits

2.6.2 Fields

- **imm[11:0]** (12 bits): a **sign-extended** immediate value (range: -2048 to $+2047$).
- **rs1** (5 bits): source register.
- **funct3** (3 bits): selects the operation.
- **rd** (5 bits): destination register.
- **opcode** (7 bits): identifies the instruction sub-category.

2.6.3 I-Type Arithmetic Instructions (`opcode = 0010011`)

Instruction	funct3	Operation	
`ADDI rd, rs1, imm`	000	$\texttt{rd} \leftarrow \texttt{rs1} + \text{sext}(\text{imm})$	
`SLTI rd, rs1, imm`	010	$\texttt{rd} \leftarrow (\texttt{rs1} <_s \text{sext}(\text{imm}))\ ?\ 1 : 0$	
`SLTIU rd, rs1, imm`	011	$\texttt{rd} \leftarrow (\texttt{rs1} <_u \text{sext}(\text{imm}))\ ?\ 1 : 0$	
`XORI rd, rs1, imm`	100	$\texttt{rd} \leftarrow \texttt{rs1} \oplus \text{sext}(\text{imm})$	
`ORI rd, rs1, imm`	110	$\texttt{rd} \leftarrow \texttt{rs1} \mid \text{sext}(\text{imm})$	
`ANDI rd, rs1, imm`	111	$\texttt{rd} \leftarrow \texttt{rs1} \,\&\, \text{sext}(\text{imm})$	
`SLLI rd, rs1, shamt`	001	$\texttt{rd} \leftarrow \texttt{rs1} \ll \text{shamt}$	imm[11:5]=0000000
`SRLI rd, rs1, shamt`	101	$\texttt{rd} \leftarrow \texttt{rs1} \gg_{\text{logic}} \text{shamt}$	imm[11:5]=0000000
`SRAI rd, rs1, shamt`	101	$\texttt{rd} \leftarrow \texttt{rs1} \gg_{\text{arith}} \text{shamt}$	imm[11:5]=0100000

Shift immediates: For SLLI, SRLI, and SRAI, only the lower 5 bits of the immediate (`shamt` = shift amount, bits [4:0]) specify the shift distance (0–31). The upper bits (imm[11:5]) act like `funct7` to distinguish SRLI from SRAI.

How to think about I-Type arithmetic: Most I-type arithmetic instructions mean "take one register value and combine it with a small constant." This is why they are heavily used for loop counters, stack-pointer updates, small address adjustments, and bit masks.

- `ADDI` is the workhorse for incrementing and decrementing values.
- There is no separate `SUBI` instruction in RV32I: subtracting a constant is just `ADDI` with a negative immediate.
- `ANDI/ORI/XORI` perform bitwise operations between `rs1` and the immediate, which makes them useful for masking or setting selected bits.

Small example: `addi x5, x5, -4` subtracts 4 from `x5`.

2.6.4 I-Type Load Instructions (`opcode = 0000011`)

Load instructions read data from memory at address `rs1` + sext(imm) and place it in `rd`.

Instruction	funct3	Operation
`LB rd, imm(rs1)`	`000`	Load byte (8-bit), **sign-extend** to 32 bits → `rd`
`LH rd, imm(rs1)`	`001`	Load halfword (16-bit), **sign-extend** to 32 bits → `rd`
`LW rd, imm(rs1)`	`010`	Load word (32-bit) → `rd`
`LBU rd, imm(rs1)`	`100`	Load byte (8-bit), **zero-extend** to 32 bits → `rd`
`LHU rd, imm(rs1)`	`101`	Load halfword (16-bit), **zero-extend** to 32 bits → `rd`

Sign-extension vs. Zero-extension: `LB/LH` are **signed**: the MSB of the loaded value is replicated to fill the upper bits, preserving the sign of the original value. `LBU/LHU` are **unsigned**: the upper bits are filled with zeros. This is critical when loading narrow data into a 32-bit register.

Width reminder: In RV32I, a **byte** is 8 bits, a **halfword** is 16 bits, and a **word** is 32 bits. The load instruction tells the processor both **how many bytes to read** and **how to fill the rest of the destination register**.

- Use `LB/LBU` for 8-bit values such as characters or packed flags.
- Use `LH/LHU` for 16-bit fields.
- Use `LW` when the full 32-bit word is needed.

In a simple RV32I implementation, it is safest to think of halfwords as 2-byte aligned and words as 4-byte aligned.

2.7 S-Type Instructions (Store)

S-Type

`opcode = 0100011`

S-type instructions **store** data from a register (`rs2`) into memory. The memory address is computed as `rs1` + sext(imm). There is **no destination register** — the immediate is split across two fields.

2.7.1 Format

imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
bits [31:25]	bits [24:20]	bits [19:15]	bits [14:12]	bits [11:7]	bits [6:0]
7 bits	5 bits	5 bits	3 bits	5 bits	7 bits

2.7.2 Why Is the Immediate Split?

The S-type immediate occupies the same bit positions as R-type's `funct7` (bits [31:25]) and `rd` (bits [11:7]). This keeps the `rs1` and `rs2` fields in exactly the same position across R, I, and S formats, simplifying the hardware register-file read logic.

2.7.3 S-Type Instruction Listing

Instruction	funct3	Operation
`SB rs2, imm(rs1)`	000	Store byte: Mem[`rs1`+sext(imm)] ← `rs2`[7 : 0]
`SH rs2, imm(rs1)`	001	Store halfword: Mem[`rs1` + sext(imm)] ← `rs2`[15 : 0]
`SW rs2, imm(rs1)`	010	Store word: Mem[`rs1` + sext(imm)] ← `rs2`[31 : 0]

Store reminder: Stores move data **from a register into memory**, so there is nothing to sign-extend or zero-extend at store time.

- `SB` writes only the lower 8 bits of `rs2`.
- `SH` writes only the lower 16 bits of `rs2`.
- `SW` writes the full 32 bits of `rs2`.

That is why RV32I has `LBU`/`LHU` for unsigned loads but no "unsigned store" instructions.
Small example: if `x6 = 0x12345678`, then `sb x6, 0(x5)` stores only `0x78` to memory.

2.8 B-Type Instructions (Branch)

B-Type

`opcode = 1100011`

B-type instructions implement **conditional branches**. They compare two registers (`rs1`, `rs2`) and, if the condition is true, jump to PC + sext(offset). Otherwise, execution continues at PC + 4. The offset is a **13-bit signed** value with the least significant bit always 0 (instructions are 4-byte aligned, offsets are multiples of 2).

2.8.1 Format

[12]	[10:5]	rs2	rs1	f3	[4:1]	[11]	opcode
bit 31	bits [30:25]	bits [24:20]	bits [19:15]	bits [14:12]	bits [11:8]	bit 7	bits [6:0]
1	6	5	5	3	4	1	7

2.8.2 Immediate Reconstruction

The 13-bit branch offset is assembled from the scattered fields:

$$\text{offset} = \{\text{imm}[12], \text{imm}[11], \text{imm}[10:5], \text{imm}[4:1], 0\}$$

Bit [0] is always 0, giving a range of ±4 KiB (−4096 to +4094).

2.8.3 B-Type Instruction Listing

Instruction	funct3	Condition (branch if true)	
`BEQ rs1, rs2, offset`	000	$\texttt{rs1} = \texttt{rs2}$	(equal)
`BNE rs1, rs2, offset`	001	$\texttt{rs1} \neq \texttt{rs2}$	(not equal)
`BLT rs1, rs2, offset`	100	$\texttt{rs1} <_s \texttt{rs2}$	(signed less than)
`BGE rs1, rs2, offset`	101	$\texttt{rs1} \geq_s \texttt{rs2}$	(signed greater or equal)
`BLTU rs1, rs2, offset`	110	$\texttt{rs1} <_u \texttt{rs2}$	(unsigned less than)
`BGEU rs1, rs2, offset`	111	$\texttt{rs1} \geq_u \texttt{rs2}$	(unsigned greater or equal)

No BLE / BGT? RISC-V omits "branch-if-less-or-equal" and "branch-if-greater-than" because they can be synthesized by swapping operands:

- `BLE rs1, rs2, offset` ≡ `BGE rs2, rs1, offset`
- `BGT rs1, rs2, offset` ≡ `BLT rs2, rs1, offset`

This keeps the hardware simple without losing expressiveness.

Branch reminder: A branch does not contain a full absolute destination address. Instead, it says "if this comparison is true, add this signed offset to the current `PC`." If the comparison is false, execution simply continues at the next instruction (`PC + 4`).

Because bit 0 of the encoded offset is always 0, the target moves in steps of 2 bytes rather than 1 byte.

Small example: if the current `PC` is `0x1000` and the branch offset is `+16`, the branch target is `0x1010`. If the condition is false, execution continues at `0x1004`.

2.9 U-Type Instructions (Upper Immediate)

U-Type

`opcodes: 0110111 (LUI), 0010111 (AUIPC)`

U-type instructions carry a **20-bit upper immediate** that is placed into bits [31:12] of the destination register. The lower 12 bits are set to zero. This format is used to construct large constants or PC-relative addresses.

2.9.1 Format

imm[31:12]	rd	opcode
bits [31:12]	bits [11:7]	bits [6:0]
20 bits	5 bits	7 bits

2.9.2 U-Type Instruction Listing

Instruction	Opcode	Operation
`LUI rd, imm`	`0110111`	`rd` $\leftarrow \text{imm}[31:12] \parallel 12'b0$ Load Upper Immediate: places the 20-bit immediate in the upper 20 bits of `rd`, zeroing the lower 12 bits.
`AUIPC rd, imm`	`0010111`	`rd` $\leftarrow \text{PC} + (\text{imm}[31:12] \parallel 12'b0)$ Add Upper Immediate to PC: adds the shifted immediate to the current PC and stores the result in `rd`. Used for PC-relative addressing of data/code.

Building 32-bit Constants: Since immediates in I-type are only 12 bits, you need **two instructions** to load a full 32-bit constant:

1. `LUI rd, upper_20` — sets bits [31:12]
2. `ADDI rd, rd, lower_12` — sets bits [11:0]

Example: load `0xDEADBEEF` into `x5`:

```
lui  x5, 0xDEADC      # x5 = 0xDEADC000
addi x5, x5, 0xEEF    # x5 = 0xDEADBEEF  (note: sign-extension adjustment)
```

LUI vs. AUIPC mental model: `LUI` builds a value from the immediate itself, while `AUIPC` builds a value **relative to the current** `PC`.

Use `LUI` when you want a large constant in a register. Use `AUIPC` when you want a base address for nearby code or data.

That is why `AUIPC` often appears with `JALR`, loads, or stores in PC-relative code sequences.

2.10 J-Type Instructions (Jump)

J-Type

`opcode = 1101111 (JAL)`

J-type provides an unconditional jump with a **21-bit signed offset** (range: ± 1 MiB). The return address ($PC + 4$) is saved in `rd`, enabling function calls.

2.10.1 Format

[20]	[10:1]	[11]	[19:12]	rd	opcode
bit 31	bits [30:21]	bit 20	bits [19:12]	bits [11:7]	bits [6:0]
1	10	1	8	5	7

2.10.2 Immediate Reconstruction

The 21-bit offset is assembled as:

$$\text{offset} = \{\text{imm}[20], \text{imm}[19:12], \text{imm}[11], \text{imm}[10:1], 0\}$$

2.10.3 Jump Instructions

Instruction	Type	Operation
`JAL rd, offset`	J-type	`rd` $\leftarrow PC + 4$; $PC \leftarrow PC + \text{sext}(\text{offset})$ Jump And Link: saves return address in `rd`, then jumps to PC-relative target.
`JALR rd, rs1, offset`	I-type	`rd` $\leftarrow PC + 4$; $PC \leftarrow$ (`rs1` $+ \text{sext}(\text{imm})$) & ~ 1 Jump And Link Register: jumps to an address computed from a register, clearing bit 0. Used for indirect jumps and function returns.

Function call / return pattern:

- **Call:** `jal ra, function_label` — jumps to function, saves return address in `ra` (`x1`).
- **Return:** `jalr zero, ra, 0` — jumps to address in `ra`, discards link (writes to `x0`).

What does "link" mean? The **link** is just the return address, usually `PC + 4`. Saving it in `ra` lets the called function know where to come back after it finishes.

If you do not need a return address, you can discard the link by writing it to `x0`. That turns `JAL` into a plain unconditional jump.

- `jal ra, function_label` — function call
- `jal x0, label` — unconditional jump without saving a return address

`JALR` is the register-based version, which is why returns are commonly written as `jalr x0, ra, 0`.

2.11 System Instructions

System

`opcode = 1110011`

The RV32I base includes two system instructions used to request services from the execution environment (OS or debugger).

Instruction	Description
`ECALL`	Environment Call: requests a service from the operating system or execution environment (e.g., a system call). The specific service is determined by the values in `a0`–`a7`.
`EBREAK`	Environment Breakpoint: transfers control to a debugger. Used to set software breakpoints during debugging.

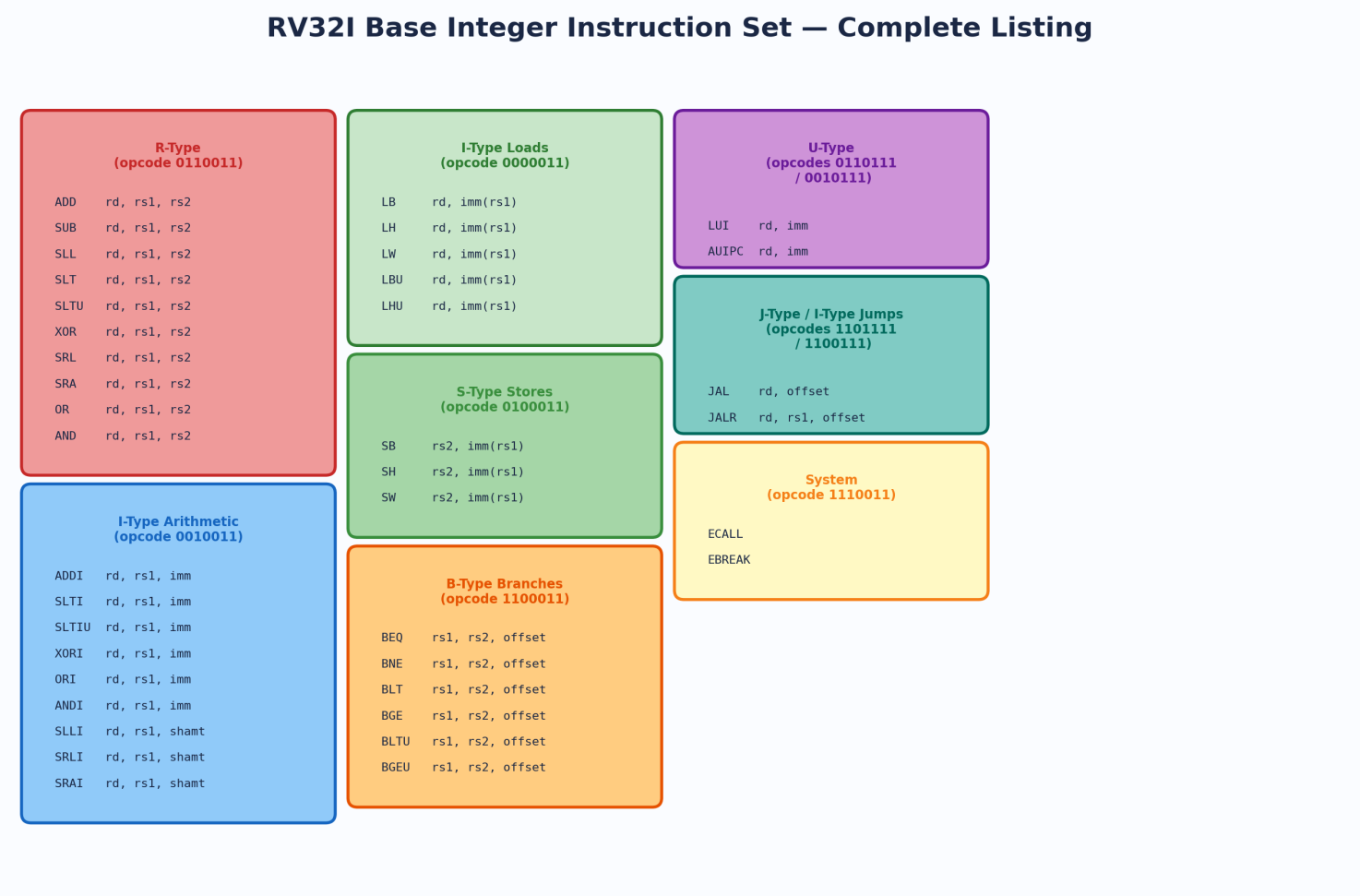
ECALL vs. EBREAK: These instructions do not behave like normal arithmetic or memory operations. Instead, they hand control to something **outside the ordinary instruction flow**.

- `ECALL` is used when the program wants the execution environment to do something on its behalf.
- `EBREAK` is used when the program wants to stop under debugger control.

The exact behavior of `ECALL` depends on the simulator, operating system, or test environment being used, while `EBREAK` is mainly a debugging aid.

3. Complete RV32I Instruction Reference

This chapter provides the **full RV32I instruction encoding table** as specified in the official RISC-V ISA manual. Every instruction is listed with its type, opcode, funct3, funct7 (where applicable), and a concise description.



3.1 Master Encoding Table

Instruction	Type	opcode	funct3	funct7/imm	Description
R-Type — Register-Register Operations (opcode 0110011)					
`ADD rd,rs1,rs2`	R	0110011	000	0000000	Add: rd = rs1 + rs2
`SUB rd,rs1,rs2`	R	0110011	000	0100000	Subtract: rd = rs1 − rs2
`SLL rd,rs1,rs2`	R	0110011	001	0000000	Shift left logical by rs2[4:0]
`SLT rd,rs1,rs2`	R	0110011	010	0000000	Set if less than (signed)
`SLTU rd,rs1,rs2`	R	0110011	011	0000000	Set if less than (unsigned)
`XOR rd,rs1,rs2`	R	0110011	100	0000000	Bitwise exclusive OR
`SRL rd,rs1,rs2`	R	0110011	101	0000000	Shift right logical by rs2[4:0]
`SRA rd,rs1,rs2`	R	0110011	101	0100000	Shift right arithmetic by rs2[4:0]
`OR rd,rs1,rs2`	R	0110011	110	0000000	Bitwise OR
`AND rd,rs1,rs2`	R	0110011	111	0000000	Bitwise AND

Continued on next page...

Instruction	Type	opcode	funct3	funct7/imm	Description
I-Type — Immediate Arithmetic (opcode 0010011)					
ADDI rd,rs1,imm	I	0010011	000	---	Add immediate
SLTI rd,rs1,imm	I	0010011	010	---	Set if $<$ imm (signed)
SLTIU rd,rs1,imm	I	0010011	011	---	Set if $<$ imm (unsigned)
XORI rd,rs1,imm	I	0010011	100	---	XOR with immediate
ORI rd,rs1,imm	I	0010011	110	---	OR with immediate
ANDI rd,rs1,imm	I	0010011	111	---	AND with immediate
SLLI rd,rs1,shamt	I	0010011	001	0000000	Shift left logical by shamt
SRLI rd,rs1,shamt	I	0010011	101	0000000	Shift right logical by shamt
SRAI rd,rs1,shamt	I	0010011	101	0100000	Shift right arithmetic by shamt
I-Type — Load Instructions (opcode 0000011)					
LB rd,imm(rs1)	I	0000011	000	---	Load byte, sign-extend
LH rd,imm(rs1)	I	0000011	001	---	Load halfword, sign-extend
LW rd,imm(rs1)	I	0000011	010	---	Load word (32-bit)
LBU rd,imm(rs1)	I	0000011	100	---	Load byte, zero-extend
LHU rd,imm(rs1)	I	0000011	101	---	Load halfword, zero-extend
S-Type — Store Instructions (opcode 0100011)					
SB rs2,imm(rs1)	S	0100011	000	---	Store byte (8-bit)
SH rs2,imm(rs1)	S	0100011	001	---	Store halfword (16-bit)
SW rs2,imm(rs1)	S	0100011	010	---	Store word (32-bit)
B-Type — Conditional Branches (opcode 1100011)					
BEQ rs1,rs2,off	B	1100011	000	---	Branch if rs1 $=$ rs2
BNE rs1,rs2,off	B	1100011	001	---	Branch if rs1 $\neq$ rs2
BLT rs1,rs2,off	B	1100011	100	---	Branch if rs1 $<$ rs2 (signed)
BGE rs1,rs2,off	B	1100011	101	---	Branch if rs1 $\geq$ rs2 (signed)
BLTU rs1,rs2,off	B	1100011	110	---	Branch if rs1 $<$ rs2 (unsigned)
BGEU rs1,rs2,off	B	1100011	111	---	Branch if rs1 $\geq$ rs2 (unsigned)
U-Type — Upper Immediate					
LUI rd,imm	U	0110111	---	---	Load upper immediate (bits [31:12])
AUIPC rd,imm	U	0010111	---	---	Add upper immediate to PC
J-Type / I-Type — Jump Instructions					
JAL rd,offset	J	1101111	---	---	Jump and link (PC-relative)
JALR rd,rs1,off	I	1100111	000	---	Jump and link register (indirect)
System Instructions (opcode 1110011)					
ECALL	I	1110011	000	000000000000	Environment call (syscall)
EBREAK	I	1110011	000	000000000001	Environment breakpoint

3.2 Instruction Count Summary

Category	Count	Instructions
R-Type (Register)	10	ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND
I-Type (Imm. Arith.)	9	ADDI, SLTI, SLTIU, XORI, ORI, ANDI, SLLI, SRLI, SRAI
I-Type (Load)	5	LB, LH, LW, LBU, LHU
S-Type (Store)	3	SB, SH, SW
B-Type (Branch)	6	BEQ, BNE, BLT, BGE, BLTU, BGEU
U-Type (Upper Imm.)	2	LUI, AUIPC
J-Type (JAL)	1	JAL
I-Type (JALR)	1	JALR
System	2	ECALL, EBREAK
Total	**39**	

Design Implication: These 39 instructions (often quoted as "40 instructions" when including FENCE) represent the *entire* base RV32I ISA. From a hardware designer's perspective, each instruction translates to a specific combination of control signals. Understanding every instruction's encoding and semantics is the foundation for designing the datapath, control unit, and decoder of the YARP processor.

4. Instruction Details & Examples

This chapter provides worked examples showing how each instruction type is encoded as a 32-bit machine word and what it does conceptually.

4.1 R-Type Example: ADD x5, x6, x7

0000000	00111	00110	000	00101	0110011
funct7	x7	x6	ADD	x5	OP

Binary: 0000000 00111 00110 000 00101 0110011
Hex: 0x007302B3
Semantics: x5 ← x6 + x7. Reads registers x6 and x7, computes their sum, and writes the result to x5.

4.2 I-Type Example: ADDI x10, x0, 42

000000101010	00000	000	01010	0010011
imm = 42	x0	ADDI	x10	OP-IMM

Semantics: x10 ← x0 + 42 = 0 + 42 = 42. This is the standard way to load a small constant into a register.

4.3 S-Type Example: SW x15, 8(x2)

Store word: writes the contents of x15 to memory at address x2 + 8.

0000000	01111	00010	010	01000	0100011
imm[11:5]=0	x15	x2	SW	imm[4:0]=	STORE

Semantics: Mem[x2 + 8] ← x15

4.4 B-Type Example: BEQ x1, x2, 16

Branch to PC + 16 if x1 equals x2.
Offset = 16: binary 0 0000001000 0 → imm[12:1] = 0_000000_1000_0
The scattered immediate fields encode: imm[12]=0, imm[11]=0, imm[10:5]=000000, imm[4:1]=1000.
Semantics: If x1 = x2, then PC ← PC + 16; else PC ← PC + 4.

4.5 U-Type Example: LUI x5, 0xDEADC

Semantics: x5 ← 0xDEADC ≪ 12 = 0xDEADC000. Places the upper 20 bits directly.

4.6 J-Type Example: JAL ra, 100

Semantics:

1. `ra` $\leftarrow$ PC + 4 (save return address)
2. PC $\leftarrow$ PC + 100 (jump to target)

This is a function call: the processor remembers where to return (in `ra`) and jumps to the function body that is 100 bytes away.

4.7 Common Pseudoinstructions

RISC-V assemblers recognize **pseudoinstructions** — mnemonics that don't correspond to a single hardware instruction but are expanded into one or more real instructions:

Pseudo	Expansion	Meaning
`nop`	`addi x0, x0, 0`	No operation
`li rd, imm`	`lui + addi`	Load 32-bit immediate
`mv rd, rs`	`addi rd, rs, 0`	Copy register
`not rd, rs`	`xori rd, rs, -1`	Bitwise NOT
`neg rd, rs`	`sub rd, x0, rs`	Two's complement negate
`j offset`	`jal x0, offset`	Unconditional jump (no link)
`ret`	`jalr x0, ra, 0`	Return from function
`call label`	`auipc + jalr`	Far function call
`beqz rs, off`	`beq rs, x0, off`	Branch if zero
`bnez rs, off`	`bne rs, x0, off`	Branch if not zero
`bgt rs,rt,off`	`blt rt, rs, off`	Branch if greater than
`ble rs,rt,off`	`bge rt, rs, off`	Branch if less or equal

Why pseudoinstructions matter: Pseudoinstructions make assembly code easier to read and write, while keeping the hardware simple. The assembler transparently expands them. For verification, only the **real** (base) instructions need to be implemented in hardware.

5. YARP RTL Implementation

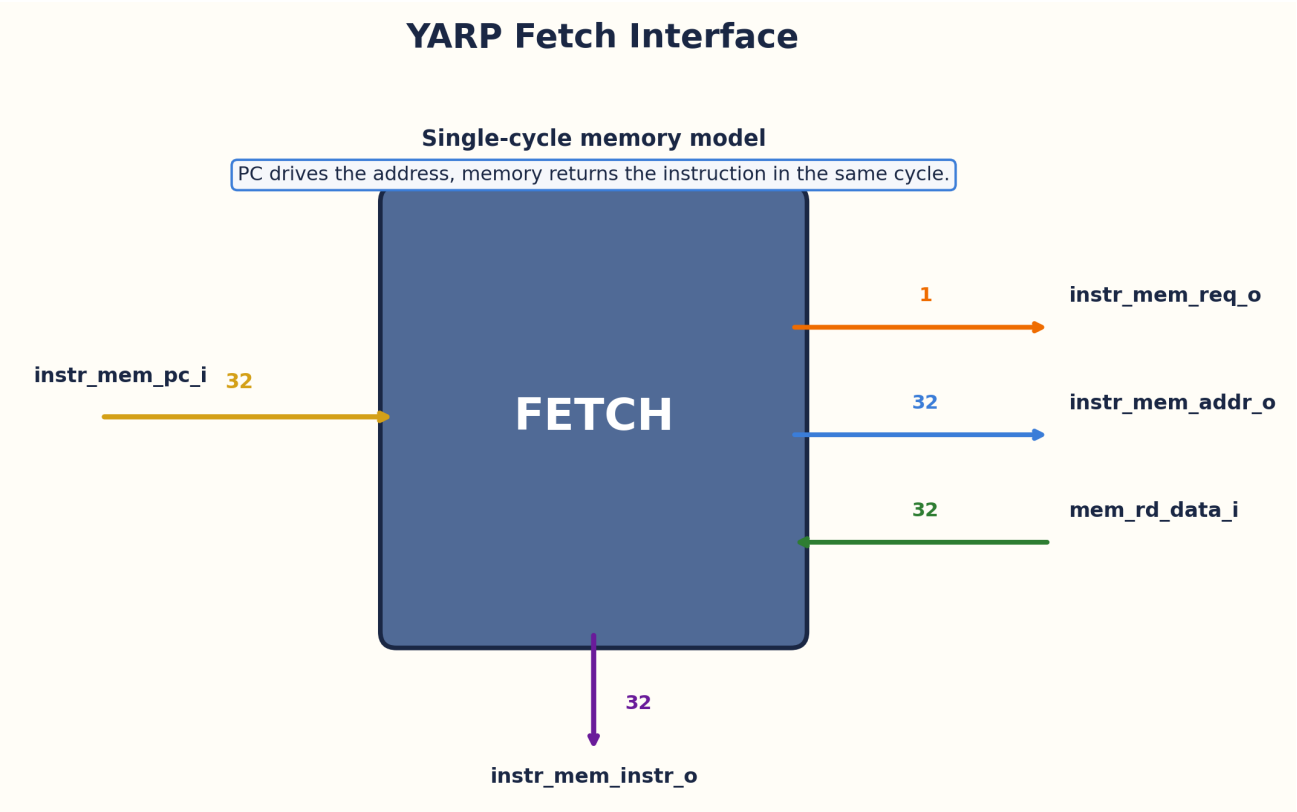
The previous chapters focused on the ISA: what instructions exist, how they are encoded, and what behavior software expects. This chapter now switches to RTL and starts building YARP from the first hardware step in the execution flow: instruction fetch.

5.1 Step 1: Fetch

Instruction memory is one of the basic building blocks of a RISC-V processor. Before the core can decode or execute an instruction, it must fetch the 32-bit instruction word from memory using the current Program Counter (PC).

Modern processors often fetch multiple instructions every cycle, but YARP is a single-cycle RV32I core. In this version of the design, the processor asks for one instruction every cycle after reset is released. The memory model is intentionally simple: a read request and address are presented to memory, and the instruction data returns in the same cycle.

Single-cycle assumption: Because the memory responds in the same cycle, there is no ready/valid handshake in this first version of YARP. The fetch block only needs to issue the request, drive the address, and forward the returned 32-bit instruction to the decode stage.



5.1.1 Design Requirements

The fetch block is intentionally minimal and follows four simple rules:

- It accepts the current PC on `instr_mem_pc_i`.
- It asserts `instr_mem_req_o` once the processor comes out of reset.

- It forwards the PC value to memory on `instr_mem_addr_o`.
- It forwards the returned memory word on `instr_mem_instr_o`.

5.1.2 Interface Definition

Signal	Dir.	Width	Description
`instr_mem_pc_i`	Input	32	Current Program Counter value used as the instruction address.
`instr_mem_req_o`	Output	1	Read request issued to instruction memory after reset is released.
`instr_mem_addr_o`	Output	32	Address sent to instruction memory. In this RTL it is a direct copy of the PC.
`mem_rd_data_i`	Input	32	Read data returned by instruction memory in the same cycle as the request.
`instr_mem_instr_o`	Output	32	Instruction word forwarded to the decode stage.

Why there is no ready signal: The current YARP memory model is zero-wait-state. That means the fetch block does not need to stall, buffer, or retry requests. Later versions that connect to a slower memory would need an explicit handshake.

5.1.3 RTL Implementation

The RTL for the fetch block is shown below:

```
module yarp_instr_mem (
  input    logic          clk,
  input    logic          reset_n,

  input    logic [31:0]   instr_mem_pc_i,

  output   logic          instr_mem_req_o,
  output   logic [31:0]   instr_mem_addr_o,

  input    logic [31:0]   mem_rd_data_i,

  output   logic [31:0]   instr_mem_instr_o
);

logic instr_mem_req_q;

always_ff @(posedge clk or negedge reset_n)
  if (!reset_n)
    instr_mem_req_q <= 1'b0;
  else
    instr_mem_req_q <= 1'b1;

assign instr_mem_req_o   = instr_mem_req_q;
assign instr_mem_addr_o  = instr_mem_pc_i;
assign instr_mem_instr_o = mem_rd_data_i;
```

```
endmodule
```

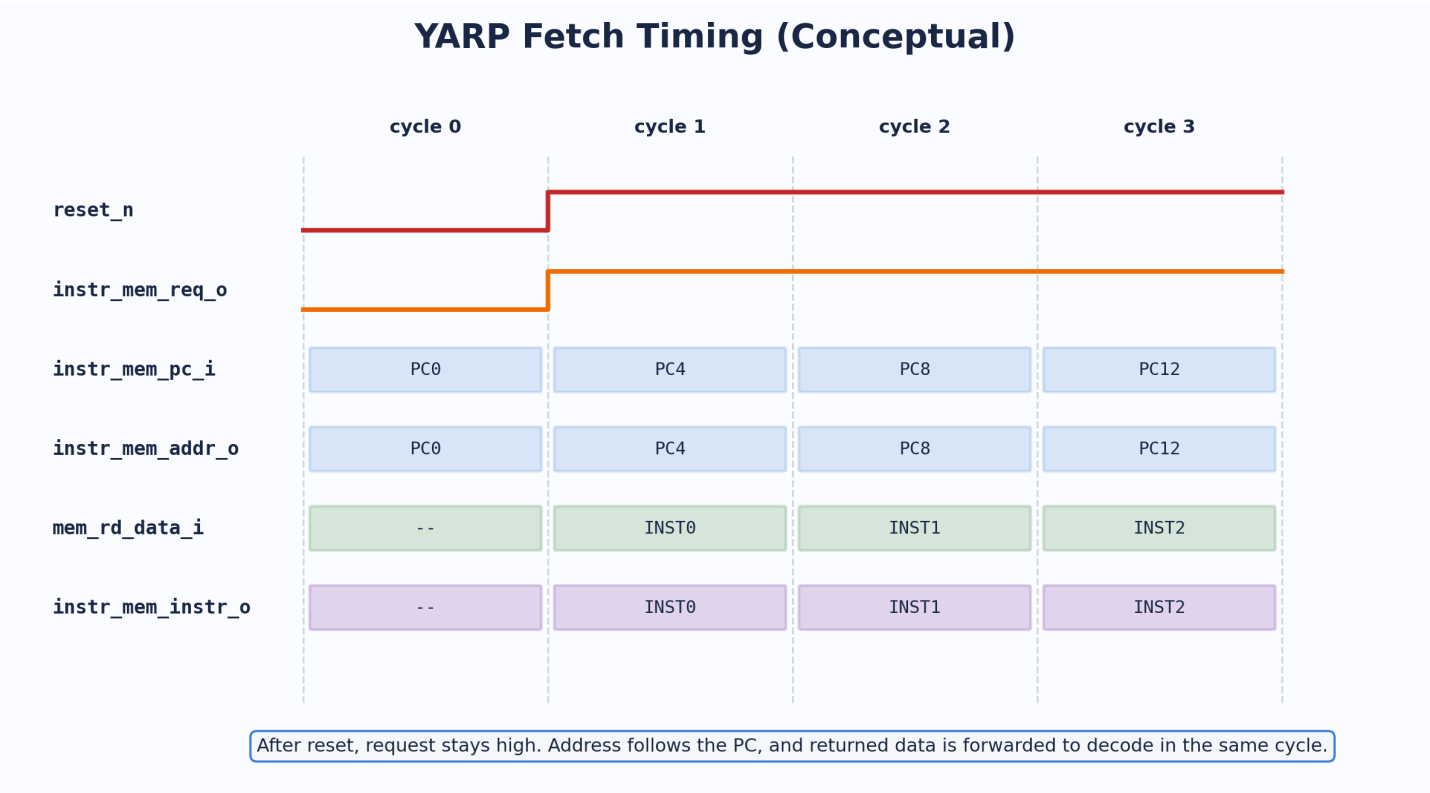
This implementation uses only one piece of sequential state: `instr_mem_req_q`. That register clears to `0` during reset and becomes `1` on the first active clock edge after reset is deasserted.

The rest of the block is purely combinational:

- `instr_mem_addr_o` always mirrors `instr_mem_pc_i`.
- `instr_mem_instr_o` always mirrors `mem_rd_data_i`.
- Once `instr_mem_req_o` goes high, it stays high, which matches a core that continuously fetches one instruction per cycle.

Important implementation detail: This fetch block does not contain the Program Counter itself. The PC is generated elsewhere in the processor and is only consumed here. The fetch block is therefore a thin interface layer between the PC path and instruction memory.

5.1.4 Cycle-by-Cycle Behavior



The timing diagram highlights the intended behavior:

1. During reset, `instr_mem_req_o` is low.
2. On the first clock after reset is released, the request goes high.
3. The PC value is driven directly onto the memory address bus.
4. Because memory responds in the same cycle, `mem_rd_data_i` and `instr_mem_instr_o` carry the same instruction word.

Fetch in one sentence: YARP fetch is a small RTL wrapper that raises a memory request after reset, sends the current PC to memory, and forwards the returned 32-bit instruction to decode without any extra buffering.